

Sura - Prueba técnica Analista de Datos y Analítica

Jorge Mario Zapata Posada

Sección 1 - Power BI avanzado: modelo semántico, storytelling y criterio

1.1. Modelo semántico como capa compartida

Usted recibe un modelo semántico en Power BI / Microsoft Fabric conectado a tablas de una fuente heredada. En el futuro, esas tablas migrarán a un Lakehouse. Su responsabilidad es revisarlo, evaluar si está bien construido y mantenerlo como capa compartida.

1.1.1. ¿Qué aspectos revisaría en un modelo semántico existente para evaluar si está bien construido? Liste al menos 6 criterios con justificación.

- El modelo debería seguir un **esquema estrella (star schema)**, en lugar de copo de nieve o de tablas planas. Dado que para este caso existen múltiples procesos de negocio (casos, facturación, prevención), lo correcto sería un **esquema de galaxia (fact constellation)**: varias tablas de hechos que comparten dimensiones conformadas, como clientes. Esto exige especial cuidado en la direccionalidad de las relaciones, para que un filtro aplicado en una tabla de hechos no se propague de forma no deseada hacia otra a través de la dimensión compartida.
- Que tenga **relaciones activas únicas y direccionalidad correcta**. Se deben evitar relaciones bidireccionales en la medida de lo posible, y usarlas solo cuando haya una razón de negocio explícita, para prevenir ambigüedad en el filtrado y problemas de rendimiento.
- **Granularidad documentada y consistente en cada tabla de hechos**. Por ejemplo, la tabla 'casos' tiene un registro por caso individual, mientras que la tabla 'facturación' tiene un registro por cliente y período. Esta granularidad debe quedar documentada explícitamente en la descripción de cada tabla, de lo contrario las medidas pueden calcularse de forma incorrecta sin que nadie lo note.
- **Medidas centralizadas en una tabla de medidas separada (_Medidas o _KPIs)**, en lugar de columnas calculadas dispersas entre las tablas. Esto facilita la gobernanza del modelo y que cualquier persona del equipo encuentre fácilmente los indicadores disponibles.

- **Nomenclatura de negocio** con nombres claros y entendibles por cualquier usuario sin conocimiento de SQL o DAX. Por ejemplo, es mucho más claro 'Costo Total de Casos' que 'sum_costo_ca'.
- **Formato consistente a nivel de medida:** los formatos de número, moneda y porcentaje deben definirse una sola vez en cada medida, en lugar de configurarse manualmente en cada visual.
- **Integridad referencial:** uso correcto de tipos de dato y ausencia de duplicados o valores huérfanos en las claves de relación. Si existieran filas huérfanas (por ejemplo, casos con un id_cliente que no existe en la tabla clientes), es una señal de un problema de integridad que debe resolverse en el origen antes de confiar en el modelo.
- **Seguridad y permisos definidos dentro del modelo:** seguridad a nivel de fila (RLS) implementada directamente en el modelo semántico, y el dataset marcado como 'Certified' para señalar que es la fuente oficial de la organización, respaldada por un proceso formal de gobierno de datos.

1.1.2. ¿Qué diferencia hay entre un modelo semántico en Power BI Desktop, en Power BI Service y en Microsoft Fabric? ¿Cuál usaría como capa compartida y por qué?

- **Power BI Desktop:** El modelo semántico existe únicamente como un objeto local dentro de un archivo .pbix. No tiene identidad propia fuera de ese archivo: no se puede compartir, no se puede conectar otro reporte a él, no tiene actualización automática de datos. Tampoco tiene ningún tipo de gobernanza realmente activa (aunque se puedan definir roles de RLS, no se aplican a ningún otro usuario, ya que el archivo no se comparte).
- **Power BI Service:** Al ser publicado, el modelo semántico se convierte en un objeto independiente y persistente dentro de un workspace (dataset), desacoplado del reporte visual que lo acompaña. Esto le da tres capacidades que no tenía en Desktop:
 - **Reutilización:** otros reportes pueden conectarse a ese mismo modelo sin necesidad de duplicarlo ('Use existing dataset').
 - **Actualización programada,** para que se refresque solo, sin depender de que alguien lo abra en Desktop.
 - **Gobernanza activa:** El RLS ahora sí se aplica realmente a cada usuario que abre un reporte conectado a él, y se le puede aplicar una etiqueta de 'Promoted' o 'Certified'.

- **Microsoft Fabric:** El modelo semántico da un salto adicional y pasa a ser un elemento del mismo lago de datos (OneLake) donde vive el resto de la plataforma, no solo un objeto dentro de un workspace de BI. Esto significa que:
 - Puede conectarse en modo DirectLake directamente a las tablas Delta del Lakehouse, sin necesidad de importar los datos ni de hacer DirectQuery, obteniendo datos casi en tiempo real con rendimiento cercano a Import.
 - Queda expuesto al mismo catálogo de gobernanza que el resto de los datos de la organización (linaje, sensibilidad, certificación), no solo al catálogo de Power BI.
 - Puede ser consumido no solo por reportes de Power BI, sino potencialmente por notebooks o consultas SQL que necesiten las mismas definiciones de negocio.

Usaría como capa compartida el modelo semántico publicado en Fabric, en modo DirectLake, precisamente porque es el único de los tres donde el modelo deja de ser solo un dataset de BI y pasa a ser una pieza más del ciclo de vida único del dato, evitando que existan dos copias (una en el Lakehouse, otra importada dentro del modelo) que se puedan desincronizar con el tiempo.

1.1.3. Cuando las tablas migren de la fuente heredada al Lakehouse, ¿qué impacto tendría sobre el modelo semántico y cómo lo manejaría para que los tableros no se rompan?

El riesgo principal no es que los datos cambien de lugar, sino que cualquier diferencia entre la fuente heredada y el Lakehouse se propague silenciosamente hasta los tableros. Los impactos más relevantes son:

- Cambios en nombres de tabla/columna o en tipos de dato que rompen relaciones y medidas DAX.
- Cambios silenciosos en granularidad o en reglas de negocio del origen (por ejemplo, si el pipeline nuevo deja de excluir registros de prueba que la fuente heredada sí filtraba).
- Cambios en el modo de conexión (de Import o DirectQuery a DirectLake), que pueden alterar el comportamiento de ciertas funciones de time intelligence.

La estrategia para manejarlo es tratar el modelo semántico como un contrato de interfaz que no debe cambiar, absorbiendo toda la complejidad de la migración en la capa de datos. En la práctica:

- Se documenta primero el contrato actual del modelo (tablas, columnas, tipos de dato y granularidad esperados)

- Se construye una capa Gold en el Lakehouse que replique exactamente ese contrato, de forma que el único cambio real sea repuntar el origen de la conexión, sin reescribir relaciones ni medidas
- Se migra en un workspace paralelo (Dev) y se promueve a producción mediante Deployment Pipelines (Dev > Test > Prod)
- Se valida con una batería de medidas de control, comparando los totales clave entre el modelo viejo y el nuevo para el mismo período, manteniendo ese monitoreo activo también después de la migración para detectar cualquier desviación no vista en la validación inicial.

1.1.4. Cómo garantizar que las medidas DAX del modelo son consistentes y que dos áreas no calculan el mismo indicador de forma diferente

- Centralizar todas las medidas en la tabla _Medidas del modelo compartido, organizada con carpetas de visualización por dominio (Casos, Financieros, Prevención), y restringir los permisos de edición del modelo únicamente al equipo central de gobierno de datos. Las demás áreas consumen el modelo mediante Live Connection, sin poder duplicarlo ni modificar sus medidas.
- Prohibir que las áreas creen modelos propios conectándose directamente a las tablas fuente. Si un área necesita un indicador nuevo, el camino correcto es solicitarlo al equipo de gobierno para que se agregue al modelo compartido, no construir su propia versión desde cero.
- Mantener un diccionario de métricas como fuente de verdad, con nombre de negocio, fórmula DAX exacta, definición en lenguaje de negocio, dueño del indicador y fecha de última validación. Esto resuelve no solo diferencias de sintaxis, sino también diferencias de interpretación del concepto de negocio, que es la causa más común de que dos áreas calculen el mismo indicador de forma distinta.
- Certificar el modelo semántico compartido como *Certified* en lugar de solo *Promoted*, para dar una señal organizacional clara de que ese es el modelo oficial, reduciendo el incentivo de crear copias propias.
- Auditar el modelo periódicamente (por ejemplo, de forma trimestral) comparando las medidas contra el diccionario documentado, apoyándose en herramientas como Tabular Editor o Best Practice Analyzer para detectar medidas duplicadas, sin formato definido o con lógica inconsistente.
- Definir un canal formal para solicitar nuevas medidas o cambios a las existentes, de forma que sea más rápido pedir las al equipo de gobierno que crear una versión propia por fuera del modelo.

1.2. Dataflows y fuente única

El equipo tiene 7 reportes que calculan la tasa de incidencia de formas diferentes. Nadie sabe cuál es el correcto y los números no coinciden en las reuniones de gerencia.

1.2.1. ¿Cómo usaría los Dataflows de Power BI para resolver este problema de raíz? ¿Dónde viviría la definición oficial de tasa de incidencia?

El primer paso sería un diagnóstico: revisar los 7 reportes para identificar en qué difiere exactamente el cálculo (por ejemplo, distinto criterio de "trabajador activo", distinto manejo de casos anulados, o distinto umbral para considerar un caso "grave"). Luego, junto con el área de negocio dueña del indicador (no el equipo de datos), se define una única fórmula oficial de tasa de incidencia.

Esa definición se materializa en un Dataflow (por ejemplo, `dfg_tasa_incidencia`), que parte directamente de las tablas fuente (casos, facturación), aplica los filtros y agregaciones acordados, y entrega una tabla limpia y reutilizable con la tasa ya calculada por cliente y período. Los 7 reportes se reconectan a esa misma entidad del Dataflow, en lugar de calcular la tasa cada uno por su cuenta desde las tablas crudas, eliminando físicamente la posibilidad de que existan versiones distintas. El Dataflow además se certifica, para que cualquier persona que construya un reporte nuevo sepa que ese es el punto de partida oficial.

La definición oficial vive en dos capas con propósitos distintos: la lógica de transformación vive en el Dataflow (qué se considera "activo", cómo se calculan numerador y denominador), que es donde se resuelve la ambigüedad de negocio. La fórmula final que arma la razón y la muestra con el formato correcto vive como una medida DAX en la tabla `_Medidas` del modelo semántico, tomando como base la tabla ya limpia que entrega el Dataflow. Esta separación es clave porque, si toda la lógica viviera solo en DAX, cualquier modelo que se conecte directamente a las tablas crudas podría recrear su propia versión del cálculo y el problema original se repetiría.

1.2.2. ¿Cuándo preferiría que esa lógica viviera en el Lakehouse en lugar del Dataflow? ¿Qué factores consideraría?

Un Dataflow y una transformación en el Lakehouse (vía notebook de Spark o un pipeline de Fabric Data Engineering) hacen esencialmente lo mismo: convierten datos crudos en algo limpio y reutilizable. La diferencia no está en qué hacen, sino en dónde vive el motor de

cómputo, quién lo mantiene, y qué tan compleja o pesada es la transformación. Los factores que consideraría son:

- Volumen de datos: si la transformación debe procesar grandes volúmenes históricos, el motor de Power Query (Dataflow) se vuelve lento y costoso, mientras que Spark, al ser un motor distribuido, escala mucho mejor.
- Complejidad de la lógica: para reglas de negocio simples (filtros, agrupaciones, ratios), un Dataflow es suficiente y más rápido de construir. Para lógica compleja, como ventanas de tiempo avanzadas o cruces pesados con múltiples fuentes, un notebook de Spark da más control y rendimiento.
- Quién necesita reutilizar el resultado, y desde dónde: si la tabla resultante solo la consume Power BI, un Dataflow es suficiente. Si otros procesos de la organización (notebooks de ciencia de datos, otros pipelines) también necesitan la misma definición, conviene que viva como una tabla Gold en el Lakehouse, disponible para cualquier proceso a través de OneLake, sin depender de que Power BI la calcule primero.
- Quién mantiene la lógica en el día a día: si el mantenimiento lo hace el equipo de BI, un Dataflow reduce fricción operativa. Si ya existe un equipo de ingeniería de datos manteniendo pipelines de Spark, conviene consolidar ahí en lugar de mantener dos stacks de transformación distintos.
- Necesidad de versionado y control de cambios: las transformaciones en el Lakehouse se pueden versionar en Git de forma nativa, con revisión de código y CI/CD, algo más limitado en los Dataflows tradicionales. Para un indicador crítico para toda la compañía, ese control adicional pesa a favor del Lakehouse.

Para el caso específico de los 7 reportes con tasa de incidencia inconsistente, optaría inicialmente por un Dataflow, ya que el problema es principalmente de gobernanza (una sola definición oficial) y no de volumen o complejidad, y permite iterar rápido mientras el área de negocio termina de acordar la definición exacta. Dejaría la puerta abierta a migrar esa lógica al Lakehouse más adelante si el volumen de datos crece de forma importante, si otras áreas fuera de Power BI empiezan a necesitar la misma definición, o si la organización decide centralizar ahí toda la lógica de transformación como estándar.

1.2.3. ¿Cómo le explicaría a un gerente, sin usar términos técnicos, por qué ahora todos los reportes darán el mismo número?

Le explicaría la situación con una analogía sencilla: lo que pasaba hasta ahora es como tener 7 personas distintas cocinando la misma receta, cada una con su propia versión

escrita a mano, con pequeñas diferencias que nadie más conocía. Por eso el resultado les salía distinto a cada una, aunque todas creyeran estar haciendo lo mismo. No es que alguien lo estuviera calculando mal a propósito, es que nunca existió una única versión oficial de esa fórmula que todos usaran.

La solución no es corregir reporte por reporte, sino definir junto con el área de negocio cuál es la fórmula correcta una sola vez, dejarla guardada en un único lugar central, y hacer que los 7 reportes tomen ese número ya calculado desde esa misma fuente, en lugar de calcularlo cada uno por su lado. De esa forma es imposible que sigan dando números distintos, porque ya no hay 7 cálculos separados, hay uno solo que todos comparten.

Adicionalmente, ese número quedaría marcado como la fuente oficial certificada dentro de la plataforma, para que cualquier reporte nuevo que se construya en el futuro parta automáticamente de ahí, y este problema no vuelva a presentarse con un octavo reporte.

1.3. Storytelling y tableros que se explican solos

En el cargo construirá un tablero de control ejecutivo que debe poder ser leído e interpretado por cualquier persona de la organización sin que nadie se lo explique.

1.3.1. ¿Qué principios seguiría para construir un tablero que cuente una historia coherente de principio a fin? Mencione al menos 5 principios concretos.

Seguiría los siguientes principios:

- **Jerarquía visual descendente**, de lo general a lo específico: el tablero debe organizarse como una pirámide invertida, con lo más importante arriba y a la izquierda (donde el ojo lee primero), y el detalle en páginas o secciones secundarias. Un gerente debe poder captar el mensaje principal en pocos segundos, sin hacer scroll ni cambiar de página.
- **Una idea central por página**, evitando la sobrecarga cognitiva: cada página debe responder una sola pregunta de negocio (por ejemplo, "¿cómo estamos en general este mes?" o "¿qué clientes necesitan atención?"), en lugar de mezclar varias preguntas distintas en el mismo espacio, lo cual rompe el hilo narrativo del tablero.
- **Contexto siempre visible**, ningún número sin punto de comparación: todo KPI debe mostrarse junto a una referencia que le dé significado, como la variación contra el período anterior o contra el promedio histórico. Un número aislado, como "142 casos", no comunica si eso es bueno o malo por sí solo.

- **Uso consistente e intencional del color**, como lenguaje y no como decoración: el color debe reservarse para comunicar estado o urgencia (por ejemplo, rojo para crítico, ámbar para moderado, verde para bajo), de forma idéntica en todo el tablero y en todos los reportes de la organización. Si el mismo color significa cosas distintas en páginas distintas, el lector pierde la capacidad de interpretar de un vistazo.
- **Anotaciones y líneas de referencia directamente sobre el gráfico**: cuando algo relevante ocurre en una serie de tiempo (un pico inusual, el lanzamiento de una campaña), debe anotarse directamente en el gráfico, no explicarse aparte en un párrafo que el lector tenga que cruzar mentalmente con la imagen.

1.3.2. ¿Cómo decidiría qué métricas van en la primera página vs en páginas de detalle?

El criterio central es hacerme una pregunta concreta para cada métrica candidata: ¿esta métrica, por sí sola, le permite a un gerente saber si debe preocuparse o actuar hoy, sin necesitar ningún otro dato adicional? Si la respuesta es sí, va en la primera página; si depende de otro dato o solo es útil cuando ya se identificó un problema, va en una página de detalle. Este criterio se apoya en varios filtros concretos:

- Nivel de agregación: la primera página muestra números agregados a nivel organizacional (totales, tasas generales), nunca el detalle de una entidad individual como un cliente específico, que corresponde a una página secundaria.
- Frecuencia de acción vs. frecuencia de monitoreo: una métrica que se revisa a diario para decidir si algo requiere acción inmediata va en portada; una métrica que solo se consulta para investigar la causa de un problema ya identificado va en detalle.
- Autonomía de interpretación: una métrica va en portada solo si se puede interpretar sin cruzarla con otras (por ejemplo, una tasa de incidencia ya clasificada como crítica, moderada o baja). Si solo cobra sentido comparada con varias métricas más, pertenece a una página de análisis detallado.
- Audiencia objetivo de cada página: la primera página se diseña para la audiencia más amplia y con menos tiempo disponible; las páginas de detalle, para roles específicos que sí necesitan profundizar (por ejemplo, un coordinador que revisa cliente por cliente para planear visitas).
- Regla práctica de cantidad: máximo 5 a 7 elementos visuales en la primera página, ya que más allá de ese número el lector empieza a perder capacidad de procesar todo en un solo vistazo.

Aplicado al tablero ejecutivo de este caso de negocio, en la primera página incluiría los 3 indicadores clave con su variación (total de casos, costo total, tasa de incidencia), el gráfico

de tendencia mensual con línea de referencia, y un indicador simple del número de clientes en estado crítico. En páginas de detalle incluiría la tabla de los 10 clientes con más casos y su clase de riesgo, la distribución de casos por tipo y por clase de riesgo, y el detalle de actividades de prevención por cliente.

1.3.3. Un gerente le dice que el tablero 'se ve bonito pero no le sirve para tomar decisiones'.
¿Qué preguntas le haría y qué cambiaría?

Antes de rediseñar el tablero, le haría preguntas puntuales para identificar la causa real detrás del comentario:

- ¿Cuándo fue la última vez que lo usaste para tomar una decisión concreta? Así se busca un ejemplo real: si no lo hay, el problema probablemente no es de diseño, sino que el tablero no está integrado en su rutina de trabajo.
- ¿Qué decisión específica intentabas tomar en ese momento? Esta pregunta suele revelar que el tablero se diseñó alrededor de qué datos existen, en lugar de qué decisión necesita tomar el gerente.
- ¿Falta información, o la información está pero no la encuentras rápido? Distingue un problema de contenido (falta el dato) de un problema de diseño o jerarquía (el dato está, pero mal ubicado o sin contexto).
- ¿El tablero te dice qué hacer, o solo te dice qué pasó? Detecta si el tablero describe bien el pasado, pero no ofrece ninguna pista de hacia dónde mirar después.
- ¿Hay algún número en el tablero del que no confíes completamente? Detecta si el verdadero problema es de confianza en el dato (como el caso de la tasa de incidencia inconsistente), y no de diseño visual.

Según lo que revelen las respuestas, los cambios serían distintos:

- Si el tablero no está alineado a una decisión específica, lo rediseñaría partiendo explícitamente de esa decisión.
- Si el problema es de jerarquía, movería esa información a la primera página o le daría más peso visual sin rediseñar todo
- Si el problema es de confianza en los datos, la solución pasa por la gobernanza y certificación de la fuente, no por el diseño
- Si el tablero solo describe el pasado, agregaría elementos que resalten automáticamente qué revisar (como los clientes en estado crítico), en lugar de dejar que el gerente lo infiera por su cuenta.

1.4. Acceso por perfil de usuario

Los coordinadores de las cuentas empresariales, deben ver solo los clientes que tienen asignados. Hay 60 coordinadores y las asignaciones cambian con frecuencia.

1.4.1. ¿Implementaría seguridad estática o dinámica? Justifique y describa la tabla que necesitaría en el modelo.

Implementaría seguridad dinámica (RLS dinámico), no estática. La razón está en el propio enunciado: hay 60 coordinadores y las asignaciones cambian con frecuencia. Con seguridad estática, cada coordinador requeriría su propio rol configurado manualmente en el modelo, y cada cambio de asignación implicaría editar el rol correspondiente en Power BI Desktop y volver a publicar el modelo completo, lo cual es inviable operativamente a esta escala y frecuencia de cambio. Con seguridad dinámica el cambio de asignación se resuelve actualizando una fila en una tabla de datos, sin tocar ni republicar el modelo semántico.

Para esto necesitaría una tabla puente de asignaciones dentro del modelo, por ejemplo:

Columna	Tipo	Descripción
id_asignacion	INT	Llave primaria
correo_coordinador	VARCHAR	Debe coincidir exactamente con el UserPrincipalName (correo corporativo) de Azure AD, ya que es contra ese valor que se compara USERPRINCIPALNAME() en DAX
id_cliente	INT	Llave foránea hacia la tabla clientes
fecha_inicio_asignacion	DATE	Opcional, útil para trazabilidad histórica
fecha_fin_asignacion	DATE	Opcional, NULL si la asignación sigue vigente

Esta tabla se relaciona con clientes mediante id_cliente, y se oculta del panel de reportes (*Hide in report view*), ya que es un mecanismo de seguridad y no un dato para analizar visualmente. El rol de RLS se define una sola vez con una expresión como [correo_coordinador] = USERPRINCIPALNAME() aplicada sobre esta tabla, y al estar relacionada con clientes, el filtro se propaga automáticamente a todas las tablas de hechos (casos, facturación, prevención).

Idealmente, esta tabla de asignaciones se alimentaría automáticamente desde el sistema donde ya se gestionan las asignaciones comerciales (CRM, Salesforce u otro), en lugar de

mantenerse manualmente, para evitar crear una fuente de verdad paralela que pueda desincronizarse de la información real.

1.4.2. ¿Cómo verificaría que la seguridad funciona correctamente antes de publicar?

Usaría la función *View as Roles* de Power BI Desktop, ingresando el `UserPrincipalName` (correo) de coordinadores reales, para simular exactamente lo que cada uno vería, probando el flujo completo desde el correo del usuario hasta el filtrado final en los visuales. El proceso de prueba seguiría estos pasos:

- Seleccionar una muestra representativa de coordinadores: uno con pocos clientes asignados, uno con muchos, y uno recién agregado a la tabla de asignaciones, para cubrir distintos escenarios.
- Para cada coordinador de prueba, verificar que todos sus clientes asignados aparezcan, que ningún cliente no asignado aparezca en ninguna visualización (incluyendo tablas de detalle, tooltips y totales generales), y que los KPIs reflejen la suma correcta solo de sus clientes, no el total de la compañía.
- Probar el caso de un usuario sin asignación en la tabla: el comportamiento esperado es que el reporte se muestre completamente vacío, nunca que muestre todos los clientes por defecto.
- Validar nuevamente ya publicado en el Service, idealmente con una cuenta de prueba o un colega que inicie sesión con las credenciales de un coordinador real, ya que algunos problemas de configuración de roles solo aparecen después de publicar.
- Documentar la prueba realizada (qué coordinadores se probaron, qué se esperaba ver y qué se vio), como evidencia formal de la validación de seguridad antes de publicar.

El principio general detrás de esta validación es que el sistema debe fallar cerrado y no abierto: si por cualquier motivo no se puede determinar la asignación de un usuario (un correo mal registrado, un coordinador nuevo), el resultado por defecto debe ser que no vea ningún dato, nunca que vea todos los clientes.

1.4.3. ¿Qué pasa cuando la asignación de un cliente cambia? ¿Qué debe actualizarse?

Si el cliente X pasa del coordinador Juan al coordinador María, lo único que debe actualizarse es una fila en la tabla `asignacion_coordinadores`: se cierra la fila de Juan (con `fecha_fin_asignacion`, si se lleva trazabilidad histórica) y se crea una nueva fila para María.

No se debe tocar el modelo semántico, no se debe editar ningún rol de RLS, y no es necesario volver a publicar nada. Ese es justamente el beneficio de haber optado por seguridad dinámica: el cambio de negocio queda desacoplado del ciclo de vida técnico del modelo.

Sin embargo, que ese cambio se refleje realmente depende del modo de conexión del modelo. Si está en modo Import, el cambio solo se ve reflejado después del próximo refresh programado, lo que genera una ventana de tiempo donde la asignación anterior podría seguir siendo visible. Si está en modo DirectQuery o DirectLake, el cambio se refleja prácticamente en tiempo real, ya que cada consulta lee el estado actual de la tabla de asignaciones.

Dado que las asignaciones cambian con frecuencia y el impacto de un error de acceso es sensible, consideraría que la tabla `asignacion_coordinadores` tenga una actualización más frecuente que el resto del modelo (por ejemplo, un refresh incremental cada hora, o DirectQuery solo para esa tabla), para minimizar la ventana de inconsistencia entre el cambio real de negocio y lo que ve el coordinador en el reporte.

Adicionalmente, si esta tabla se alimenta automáticamente desde el sistema donde el área comercial ya gestiona las asignaciones (como un CRM), el cambio ocurre de forma natural sin intervención del equipo de datos. Si se mantiene manualmente, sería necesario definir un responsable claro de actualizarla, para que no se convierta en un cuello de botella operativo.

1.5. Medidas DAX

Explique en términos de negocio qué calcula cada medida y cuándo la usaría. Incluya el comportamiento cuando no hay datos:

1.5.1. Medida A:

`Casos YTD = TOTALYTD(COUNT(casos[id_caso]), calendario[fecha])`

Representa el total acumulado de casos registrados desde el inicio del año hasta el período que se esté visualizando en el reporte (Year-To-Date). Por ejemplo, si el reporte está filtrado en junio, la medida no muestra los casos de junio, sino la suma de todos los casos ocurridos de enero a junio inclusive.

La usaría para responder preguntas de comparación de desempeño acumulado, tipo "¿cómo vamos este año comparado con el mismo punto del año pasado?", y sería una

métrica adecuada para la primera página de un tablero ejecutivo. No la usaría cuando se necesite ver la tendencia mes a mes, ya que un acumulado YTD solo puede subir o mantenerse igual, lo cual esconde visualmente los picos de un mes específico.

Respecto al comportamiento cuando no hay datos, hay dos escenarios distintos: si no hay ningún caso registrado en todo el año hasta la fecha evaluada, COUNT() devuelve 0 (no BLANK()), ya que contar cero filas da como resultado el número cero. En cambio, si la fecha evaluada está fuera del rango de la tabla calendario (por ejemplo, un período futuro sin datos aún, o anterior al primer registro histórico), TOTALYTD puede devolver BLANK(), al no existir una fecha válida sobre la cual acumular.

Esta distinción importa porque un 0 comunica claramente "no ha habido casos", mientras que un BLANK() (mostrado como espacio vacío) puede generar dudas sobre si realmente no hay casos o si el dato no cargó correctamente. Por eso, envolvería la medida de la siguiente forma, para forzar un 0 explícito ante cualquier escenario de BLANK():

```
Casos YTD =  
VAR resultado = TOTALYTD(COUNT(casos[id_caso]), calendario[fecha])  
RETURN COALESCE(resultado, 0)
```

1.5.2. Medida B:

```
% participación =  
DIVIDE([Total casos], CALCULATE([Total casos], ALL(clientes)), 0)
```

¿Por qué es necesario ALL(clientes) en la Medida B? ¿Qué pasaría si no estuviera?

Calcula el porcentaje que representan los casos del contexto actual (por ejemplo, un cliente, un sector o una clase de riesgo específica) sobre el total de casos de toda la compañía, sin importar qué filtro esté aplicado en el reporte. Por ejemplo, si el reporte está filtrado por un cliente específico y la medida muestra 4.2%, significa que los casos de ese cliente representan el 4.2% de todos los casos registrados en la organización en el período seleccionado.

ALL(clientes) es necesario porque, sin él, el denominador (CALCULATE([Total casos])) simplemente respetaría el mismo contexto de filtro que ya tiene el numerador, dando como resultado el mismo valor en ambos lados de la división. Esto haría que la medida siempre devolviera 100%, sin importar qué cliente estuviera filtrado, perdiendo por completo su propósito. ALL(clientes) obliga al denominador a ignorar el filtro de cliente activo y calcular el total como si se viera la compañía completa, permitiendo así una comparación real entre el contexto filtrado y el total general.

Es importante notar que ALL(clientes) elimina el filtro sobre toda la tabla clientes, incluyendo otras columnas como clase_riesgo si también estuvieran filtradas. Si la intención de negocio fuera comparar un cliente contra otros de su misma clase de riesgo (en lugar de contra toda la compañía), sería necesario usar ALL(clientes[id_cliente]) en lugar de ALL(clientes), para eliminar únicamente el filtro de cliente y mantener el de clase de riesgo activo.

La usaría en visuales donde se necesite dar contexto relativo a un número absoluto, por ejemplo junto a la columna de total de casos en la tabla de los 10 clientes con más casos, para que el lector identifique de inmediato si un cliente con muchos casos representa una porción realmente significativa de la operación total, o si simplemente es grande en términos absolutos por tener muchos trabajadores.

Respecto al comportamiento cuando no hay datos: si el cliente filtrado no tiene casos pero sí existen casos en el resto de la compañía, la medida devuelve 0% correctamente. Si no hay casos en absoluto en toda la compañía para el período filtrado, DIVIDE devuelve su tercer argumento (0), evitando el error de división por cero. Gracias a este tercer parámetro, la medida siempre devuelve un 0% explícito y nunca un valor en blanco ambiguo.

1.5.3. Medida C:

```
Var vs mes anterior =  
[Casos mes actual] - CALCULATE([Casos mes actual],  
PREVIOUSMONTH(calendario[fecha]))
```

¿Qué devuelve la Medida C cuando no hay datos del mes anterior? ¿Cómo lo manejaría?

Calcula la variación en número de casos respecto al mes inmediatamente anterior. Un valor positivo indica más casos que el mes pasado, y un valor negativo, menos casos. La usaría como indicador de tendencia de corto plazo, complementando a una medida acumulada como Casos YTD, ya que responde específicamente "¿qué ha pasado en el mes respecto al anterior?", una señal de alerta inmediata que todo KPI debería mostrar junto a su valor absoluto.

Respecto al comportamiento cuando no hay datos del mes anterior, hay dos escenarios con el mismo resultado numérico pero distinto significado de negocio. Si el mes anterior existe en el calendario pero no tuvo casos registrados, CALCULATE devuelve 0, y la resta equivale simplemente al valor del mes actual, lo cual es correcto: si no hubo actividad el mes pasado, toda la actividad de este mes es la variación real. Si en cambio el mes anterior no

existe dentro del rango de la tabla calendario (típicamente el primer mes de datos históricos disponibles), PREVIOUSMONTH no encuentra fechas válidas y CALCULATE devuelve BLANK(), que en una resta aritmética DAX trata como 0, dando el mismo resultado numérico que el escenario anterior, pero sin que exista realmente un punto de comparación válido.

Esta ambigüedad merece manejo explícito, ya que mostrar una variación "de 0 a X" en el primer mes de datos es engañoso, al no existir un mes anterior real contra el cual comparar. Por eso, la construiría de la siguiente forma, para distinguir la ausencia de mes de comparación de un mes de comparación legítimamente en cero:

```
Var vs mes anterior =  
VAR CasosMesAnterior =  
    CALCULATE([Casos mes actual], PREVIOUSMONTH(calendario[fecha]))  
VAR ExisteMesAnterior =  
    NOT ISBLANK(CALCULATE(COUNTROWS(calendario),  
        PREVIOUSMONTH(calendario[fecha])))  
RETURN  
    IF(  
        ExisteMesAnterior,  
        [Casos mes actual] - CasosMesAnterior,  
        BLANK()  
    )
```

De esta forma, en el primer mes de datos disponibles la medida devuelve explícitamente BLANK() (mostrado como espacio vacío o "N/A" en el visual), en lugar de un número que aparenta ser una variación real, evitando que se interprete como un crecimiento genuino cuando en realidad no hay historia previa con la cual comparar.

Sección 2 - SQL avanzado: construcción, pipelines y lectura de lógica

2.1. Lectura y documentación de lógica existente

Le entregan la siguiente consulta de un colega que ya no está en el equipo. Debe entenderla, identificar problemas y documentarla para publicarla en la biblioteca compartida de consultas:

```
WITH base AS (
    SELECT
        c.id_cliente, c.nombre, c.clase_riesgo,
        COUNT(ca.id_caso) AS total_casos,
        SUM(CASE WHEN ca.dias_ausencia > 15
            THEN 1 ELSE 0 END) AS casos_graves,
        SUM(ca.costo) AS costo_total,
        AVG(f.trabajadores_activos) AS trabajadores_prom
    FROM clientes c
    LEFT JOIN casos ca ON c.id_cliente = ca.id_cliente
        AND ca.fecha_ocurrencia >= DATEADD(month,-12,CURRENT_DATE)
    LEFT JOIN facturacion f ON c.id_cliente = f.id_cliente
        AND f.periodo >= DATEADD(month,-6,CURRENT_DATE)
    WHERE c.estado = 'activo'
    GROUP BY c.id_cliente, c.nombre, c.clase_riesgo
)
SELECT *,
    ROUND(casos_graves*100.0/NULLIF(total_casos,0),1) AS pct_graves,
    ROUND(costo_total/NULLIF(trabajadores_prom,0),0) AS costo_x_trab,
    DENSE_RANK() OVER (PARTITION BY clase_riesgo
        ORDER BY costo_total DESC) AS ranking_clase
FROM base WHERE total_casos > 0 ORDER BY costo_total DESC
```

2.1.1. Explique en dos párrafos, sin términos técnicos, qué está calculando esta consulta y para qué sirve

Esta consulta arma, para cada cliente activo de la compañía, un perfil resumido de lo que le ha pasado en los últimos 12 meses: cuántos incidentes tuvo, qué tan graves fueron (según cuántos implicaron más de 15 días de ausencia), y cuánto costaron en total. A eso le suma cuántos trabajadores tenía ese cliente en promedio durante los últimos 6 meses, para poder calcular cuánto le costó cada trabajador en promedio, y qué tan grave es su situación comparada con otros clientes que están en su mismo nivel de riesgo.

El resultado es una lista de clientes que sí tuvieron algún incidente en el último año, ordenada de mayor a menor costo, con una posición relativa entre los clientes de su misma clase de riesgo. En la práctica, esta consulta sirve para identificar rápidamente cuáles clientes están generando más costo o mayor proporción de casos graves, información que un analista o coordinador puede usar para decidir a quién priorizar en visitas de prevención o en revisiones de riesgo, en lugar de tener que revisar cliente por cliente manualmente.

2.1.2. Identifique al menos un problema técnico que afecta la precisión del resultado. Explíquelo y proponga la corrección.

El problema es un fan-out (explosión de filas) causado por dos LEFT JOIN paralelos de tipo “uno a muchos” hacia la misma tabla base: un cliente puede tener múltiples casos y, al mismo tiempo, múltiples registros de facturación. Al unir dos tablas de "muchos" al mismo nivel, SQL las combina como un producto cruzado: cada caso se repite una vez por cada período de facturación que también corresponda a ese cliente. Por ejemplo, un cliente con 3 casos y 6 meses de facturación generaría 18 filas antes de agrupar.

Esto infla silenciosamente COUNT(ca.id_caso) y SUM(ca.costos) por el número de meses de facturación de cada cliente (en este ejemplo, un factor de 6), sin generar ningún error visible. Además, el factor de inflación varía según cuántos meses de facturación tenga cada cliente, por lo que el error no es consistente entre clientes y distorsiona también las comparaciones (ranking_clase). El campo pct_graves no se ve afectado, porque numerador y denominador se inflan por el mismo factor y el error se cancela; tampoco se ve afectado trabajadores_prom, ya que el promedio de valores duplicados en la misma proporción da el mismo resultado que el promedio original.

La corrección consiste en pre-agregar cada tabla de hechos por separado antes de unirla con clientes, de modo que cada una llegue al JOIN final con un solo registro por cliente, evitando el producto cruzado:

```
WITH
casos_agg AS (
  SELECT
    ca.id_cliente,
    COUNT(ca.id_caso) AS total_casos,
    SUM(CASE WHEN ca.dias_ausencia > 15 THEN 1 ELSE 0 END) AS
casos_graves,
    SUM(ca.costos) AS costo_total
  FROM casos ca
```

```

        WHERE ca.fecha_ocurrencia >= DATEADD(month, -12, CURRENT_DATE)
        GROUP BY ca.id_cliente
    ),
    facturacion_agg AS (
        SELECT
            f.id_cliente,
            AVG(f.trabajadores_activos) AS trabajadores_prom
        FROM facturacion f
        WHERE f.periodo >= DATEADD(month, -6, CURRENT_DATE)
        GROUP BY f.id_cliente
    ),
    base AS (
        SELECT
            c.id_cliente, c.nombre, c.clase_riesgo,
            COALESCE(ca.total_casos, 0) AS total_casos,
            COALESCE(ca.casos_graves, 0) AS casos_graves,
            COALESCE(ca.costos_total, 0) AS costos_total,
            f.trabajadores_prom
        FROM clientes c
        LEFT JOIN casos_agg ca ON c.id_cliente = ca.id_cliente
        LEFT JOIN facturacion_agg f ON c.id_cliente = f.id_cliente
        WHERE c.estado = 'activo'
    )
    SELECT *,
        ROUND(casos_graves*100.0/NULLIF(total_casos,0),1) AS pct_graves,
        ROUND(costos_total/NULLIF(trabajadores_prom,0),0) AS costo_x_trab,
        DENSE_RANK() OVER (PARTITION BY clase_riesgo ORDER BY costos_total
        DESC) AS ranking_clase
    FROM base
    WHERE total_casos > 0
    ORDER BY costos_total DESC;

```

Cada CTE (casos_agg, facturacion_agg) agrupa su propia tabla de hechos antes de unirla con clientes, de modo que en el JOIN final cada tabla aporta como máximo una fila por cliente, lo cual elimina por completo la posibilidad de fan-out. Adicionalmente, se agregó COALESCE(..., 0) en los campos de casos, para que un cliente sin casos en el período muestre explícitamente 0 en lugar de NULL, evitando que se interprete erróneamente como un dato faltante.

2.1.3. Escriba la documentación de esta consulta en el formato que usaría para publicarla en la biblioteca compartida: nombre, propósito, tablas fuente, campos calculados y notas de uso.

Ficha de documentación – Biblioteca de consultas compartidas

****Nombre de la consulta:**** `perfil\_riesgo\_cliente\_anual`

****Propósito:**** Genera un perfil anual por cliente activo, combinando el historial de casos de los últimos 12 meses (frecuencia, severidad y costo) con el promedio de trabajadores activos de los últimos 6 meses, para priorizar la atención de coordinadores y evaluar el nivel de riesgo relativo de cada cliente dentro de su misma clase de riesgo.

****Tablas fuente:****

Tabla	Rol en la consulta
`clientes`	Tabla base; define el universo de clientes activos a analizar
`casos`	Aporta el conteo, severidad y costo de incidentes de los últimos 12 meses
`facturacion`	Aporta el promedio de trabajadores activos de los últimos 6 meses

****Campos calculados:****

Campo	Descripción	Notas
`total_casos`	Número de casos del cliente en los últimos 12 meses	0 si no tuvo casos en el período
`casos_graves`	Número de casos con más de 15 días de ausencia	Subconjunto de `total_casos`
`costo_total`	Suma del costo de todos los casos del cliente en el período	En la misma unidad monetaria que la tabla `casos`
`trabajadores_prom`	Promedio de trabajadores activos del cliente, últimos 6 meses	Puede ser NULL si el cliente no tiene registros de facturación en el período
`pct_graves`	Porcentaje de casos que fueron graves	`0` si `total_casos` es 0 (protegido con NULLIF)
`costo_x_trab`	Costo total dividido entre el promedio de trabajadores	`0` si `trabajadores_prom` es 0 o NULL (protegido con NULLIF)

| `ranking\_clase` | Posición del cliente por costo total, comparado solo con clientes de su misma clase de riesgo | 1 = mayor costo dentro de su clase; usa DENSE_RANK, por lo que puede haber empates |

****Notas de uso:****

- La consulta solo devuelve clientes con `total\_casos > 0`; clientes activos sin incidentes en el período no aparecen en el resultado.
- Las tablas de hechos (`casos`, `facturacion`) se pre-agregan por separado antes de unirse a `clientes`, para evitar un problema de duplicación de filas (fan-out) que existía en una versión anterior de esta consulta y que inflaba `total\_casos` y `costo\_total`. No modificar esta estructura de CTEs sin entender esta razón.
- Las ventanas de tiempo (12 meses para casos, 6 meses para facturación) están hardcodedas con `DATEADD`/`CURRENT\_DATE`; si el negocio requiere ventanas distintas, deben parametrizarse antes de reutilizar esta consulta en otro contexto.
- `trabajadores\_prom` puede venir NULL si el cliente no tiene facturación en los últimos 6 meses; en ese caso, `costo\_x\_trab` también será NULL (protegido por `NULLIF`, no genera error, pero debe interpretarse como "dato no disponible", no como cero).

****Última validación:**** [fecha de publicación]

****Responsable:**** [nombre del analista que la publica]

2.2. Construcción de consulta avanzada

El equipo necesita cada semana los 15 clientes que más atención prioritaria requieren para planear visitas. Construya la consulta SQL usando las tablas del contexto.

Se evalúa el criterio de priorización, que la consulta sea correcta y que los criterios tengan sentido de negocio. Justifique por escrito cada criterio.

La priorización no se basa en un solo criterio (como el costo total o el volumen de casos), sino en un score compuesto por cuatro dimensiones de riesgo distintas. La razón es que priorizar por un solo criterio tiende a favorecer siempre a los mismos clientes grandes, dejando fuera a clientes más pequeños con una tendencia de deterioro reciente, que son precisamente los casos donde una visita preventiva temprana tiene mayor impacto.

1. **Severidad y frecuencia reciente (35% del score):** se consideran los casos ocurridos en los últimos 30 días, con mayor peso a los casos graves que a los leves. Se usa una ventana corta, en lugar de 12 meses, porque el objetivo es identificar dónde está pasando algo ahora, no reconstruir el historial completo del cliente.
2. **Tendencia de la tasa de incidencia (30% del score):** se calcula la variación de la tasa de incidencia del mes actual con respecto al mes anterior, y solo se puntúa cuando la tasa empeoró. Este criterio es importante para detectar riesgo emergente: un cliente cuya tasa está subiendo, aunque todavía no sea la más alta en términos absolutos, amerita una visita antes de que el problema escale, mientras que un cliente con tasa alta pero estable probablemente ya está siendo gestionado adecuadamente.
3. **Clase de riesgo intrínseca del cliente (20% del score):** se incorpora la clasificación de riesgo (1 a 5) de cada cliente, porque el mismo nivel de actividad reciente representa un riesgo distinto según el perfil base del cliente: un repunte de casos en un cliente de riesgo 5 es más urgente que el mismo repunte en uno de riesgo 1.
4. **Ausencia de actividad de prevención reciente (15% del score):** se mide cuántos días han pasado desde la última actividad de prevención registrada, limitando el efecto a un máximo de 90 días para evitar que un cliente sin visitas desde hace años domine el score solo por antigüedad. Este criterio evita que la priorización ignore a clientes con riesgo moderado que no han sido visitados recientemente.

Los porcentajes de peso asignados (35 – 30 – 20 – 15 %) son una propuesta inicial y deberían validarse y ajustarse junto con el área de riesgo o prevención, que tiene el conocimiento de negocio para calibrar qué tanto debe pesar cada factor en la práctica.

La consulta se presenta a continuación y también está disponible como archivo SQL en el repositorio en /seccion_2/query_2_2.sql

```
WITH
casos_recientes AS (
  -- Severidad y frecuencia de los últimos 30 días
  SELECT
    id_cliente,
    COUNT(*) AS casos_30d,
    SUM(CASE WHEN tipo = 'grave' THEN 1 ELSE 0 END) AS
casos_graves_30d
  FROM casos
```

```

WHERE fecha_ocurrencia >= DATEADD(day, -30, CURRENT_DATE)
GROUP BY id_cliente
),

incidencia_actual AS (
-- Tasa de incidencia del mes en curso, para comparar tendencia
SELECT
    ca.id_cliente,
    COUNT(ca.id_caso) AS casos_mes_actual,
    AVG(f.trabajadores_activos) AS trabajadores_mes_actual
FROM casos ca
JOIN facturacion f
    ON ca.id_cliente = f.id_cliente
    AND f.periodo = DATE_TRUNC('month', CURRENT_DATE)
WHERE ca.fecha_ocurrencia >= DATE_TRUNC('month', CURRENT_DATE)
GROUP BY ca.id_cliente
),

incidencia_anterior AS (
-- Tasa de incidencia del mes anterior, para calcular tendencia
SELECT
    ca.id_cliente,
    COUNT(ca.id_caso) AS casos_mes_anterior,
    AVG(f.trabajadores_activos) AS trabajadores_mes_anterior
FROM casos ca
JOIN facturacion f
    ON ca.id_cliente = f.id_cliente
    AND f.periodo = DATE_TRUNC('month', DATEADD(month, -1,
CURRENT_DATE))
WHERE ca.fecha_ocurrencia >= DATE_TRUNC('month', DATEADD(month, -
1, CURRENT_DATE))
    AND ca.fecha_ocurrencia < DATE_TRUNC('month', CURRENT_DATE)
GROUP BY ca.id_cliente
),

ultima_prevision AS (
-- Días transcurridos desde la última actividad de prevención
SELECT
    id_cliente,

```

```

        MAX(fecha) AS fecha_ultima_prevencion,
        DATEDIFF(day, MAX(fecha), CURRENT_DATE) AS dias_sin_prevencion
FROM prevencion
GROUP BY id_cliente
),

score_final AS (
    SELECT
        c.id_cliente,
        c.nombre,
        c.clase_riesgo,
        COALESCE(cr.casos_30d, 0) AS casos_30d,
        COALESCE(cr.casos_graves_30d, 0) AS casos_graves_30d,
        COALESCE(ia.casos_mes_actual * 100.0 /
NULLIF(ia.trabajadores_mes_actual, 0), 0) AS tasa_actual,
        COALESCE(ian.casos_mes_anterior * 100.0 /
NULLIF(ian.trabajadores_mes_anterior, 0), 0) AS tasa_anterior,
        COALESCE(up.dias_sin_prevencion, 999) AS dias_sin_prevencion,
        -- Score ponderado: severidad reciente (35%) + tendencia (30%)
+ clase de riesgo (20%) + falta de prevención (15%)
        ROUND(
            (COALESCE(cr.casos_graves_30d, 0) * 10 +
COALESCE(cr.casos_30d, 0) * 3) * 0.35
            + GREATEST(
                COALESCE(ia.casos_mes_actual * 100.0 /
NULLIF(ia.trabajadores_mes_actual, 0), 0)
                - COALESCE(ian.casos_mes_anterior * 100.0 /
NULLIF(ian.trabajadores_mes_anterior, 0), 0),
                0
            ) * 0.30
            + c.clase_riesgo * 4 * 0.20
            + LEAST(COALESCE(up.dias_sin_prevencion, 999), 90) / 90.0
* 10 * 0.15
        , 2) AS score_prioridad
    FROM clientes c
    LEFT JOIN casos_recientes cr ON c.id_cliente = cr.id_cliente
    LEFT JOIN incidencia_actual ia ON c.id_cliente = ia.id_cliente
    LEFT JOIN incidencia_anterior ian ON c.id_cliente = ian.id_cliente
    LEFT JOIN ultima_prevencion up ON c.id_cliente = up.id_cliente

```

```

        WHERE c.estado = 'activo'
    )

SELECT
    id_cliente,
    nombre,
    clase_riesgo,
    casos_30d,
    casos_graves_30d,
    ROUND(tasa_actual, 2) AS tasa_incidencia_actual,
    ROUND(tasa_actual - tasa_anterior, 2) AS variacion_tasa,
    dias_sin_preencion,
    score_prioridad
FROM score_final
ORDER BY score_prioridad DESC
LIMIT 15;

```

2.3. Pipeline de datos

Construya una consulta SQL base para un pipeline diario que alimenta el tablero ejecutivo. Debe calcular para cada cliente activo, a corte del día anterior:

- Total de casos del mes en curso y variación vs el mes anterior
- Tasa de incidencia: (casos del mes / trabajadores activos) * 100
- Actividades de prevención realizadas en los últimos 30 días
- Clasificación: 'crítico' si tasa > 5, 'moderado' si entre 2 y 5, 'bajo' si menor a 2

Adicional: ¿cómo garantizaría que este pipeline detecta automáticamente cuando la calidad del dato cayó antes de que el resultado llegue al tablero?

La consulta se presenta a continuación y también está disponible como archivo SQL en el repositorio en /seccion_2/query_2_3.sql

```

WITH
parametros AS (
    SELECT
        CURRENT_DATE - 1 AS fecha_corte,
        DATE_TRUNC('month', CURRENT_DATE - 1) AS inicio_mes_actual,

```

```

        DATE_TRUNC('month', DATEADD(month, -1, CURRENT_DATE - 1)) AS
inicio_mes_anterior,
        DATEADD(month, -1, CURRENT_DATE - 1) AS
fecha_corte_mes_anterior
),

```

```

casos_mes_actual AS (
    SELECT ca.id_cliente, COUNT(*) AS casos_mes_actual
    FROM casos ca
    CROSS JOIN parametros p
    WHERE ca.fecha_ocurrencia >= p.inicio_mes_actual
        AND ca.fecha_ocurrencia <= p.fecha_corte
    GROUP BY ca.id_cliente
),

```

```

casos_mes_anterior AS (
    -- mismo número de días transcurridos que el mes actual, para una
comparación justa
    SELECT ca.id_cliente, COUNT(*) AS casos_mes_anterior
    FROM casos ca
    CROSS JOIN parametros p
    WHERE ca.fecha_ocurrencia >= p.inicio_mes_anterior
        AND ca.fecha_ocurrencia <= p.fecha_corte_mes_anterior
    GROUP BY ca.id_cliente
),

```

```

trabajadores_actuales AS (
    SELECT f.id_cliente, f.trabajadores_activos
    FROM facturacion f
    CROSS JOIN parametros p
    WHERE f.periodo = p.inicio_mes_actual
),

```

```

prevencion_30d AS (
    SELECT pr.id_cliente, COUNT(*) AS actividades_prevencion_30d
    FROM prevencion pr
    CROSS JOIN parametros p
    WHERE pr.fecha BETWEEN p.fecha_corte - 30 AND p.fecha_corte
    GROUP BY pr.id_cliente

```

)

```
SELECT
    c.id_cliente,
    c.nombre,
    c.clase_riesgo,
    COALESCE(cma.casos_mes_actual, 0) AS casos_mes_actual,
    COALESCE(cma.casos_mes_anterior, 0) -
    COALESCE(can.casos_mes_anterior, 0) AS variacion_vs_mes_anterior,
    ta.trabajadores_activos,
    ROUND(COALESCE(cma.casos_mes_actual, 0) * 100.0 /
    NULLIF(ta.trabajadores_activos, 0), 2) AS tasa_incidencia,
    COALESCE(p30.actividades_prevision_30d, 0) AS
    actividades_prevision_30d,
    CASE
        WHEN ta.trabajadores_activos IS NULL OR
        ta.trabajadores_activos = 0 THEN 'sin datos'
        WHEN COALESCE(cma.casos_mes_actual, 0) * 100.0 /
        ta.trabajadores_activos > 5 THEN 'crítico'
        WHEN COALESCE(cma.casos_mes_actual, 0) * 100.0 /
        ta.trabajadores_activos >= 2 THEN 'moderado'
        ELSE 'bajo'
    END AS clasificacion
FROM clientes c
LEFT JOIN casos_mes_actual cma ON c.id_cliente = cma.id_cliente
LEFT JOIN casos_mes_anterior can ON c.id_cliente = can.id_cliente
LEFT JOIN trabajadores_actuales ta ON c.id_cliente = ta.id_cliente
LEFT JOIN prevision_30d p30 ON c.id_cliente = p30.id_cliente
WHERE c.estado = 'activo';
```

Notas de diseño: CURRENT_DATE - 1 se usa como fecha de corte porque el pipeline corre en la madrugada procesando el día que ya cerró completamente. La condición "sin datos" en el CASE es intencional: sin ella, un cliente sin registro de facturación del mes actual generaría una división por NULL, que en la comparación NULL > 5 no es ni verdadero ni falso, y caería silenciosamente en "bajo", clasificando incorrectamente como bajo riesgo a un cliente que en realidad no tiene datos suficientes para calcular su tasa. Cada fuente se pre-agrega en su propia CTE antes del JOIN final, evitando el problema de fan-out.

¿Cómo garantizaría que el pipeline detecta automáticamente una caída en la calidad del dato? La validación de calidad debe ser un gate automático dentro del pipeline, ejecutado después de cargar los datos, pero antes de que se actualice la tabla que alimenta el tablero. De esta forma, si alguna validación falla, el pipeline puede detener la publicación de los datos y generar una alerta para su revisión.

Entre las principales validaciones incluiría:

- **Frescura:** verificar que la fecha máxima de los datos cargados corresponda al corte esperado y no se encuentre retrasada.
- **Anomalías en el volumen:** comparar el número de filas cargadas hoy contra el promedio de los últimos días. Una caída o alza superior a un umbral definido (por ejemplo 30%) se marca como sospechosa.
- **Compleitud y validez de campos críticos:** verificar que campos obligatorios, como `id_cliente`, no contengan valores nulos y que los datos cumplan las reglas de negocio definidas. Por ejemplo, `trabajadores_activos` no debería tener valores negativos y `clase_riesgo` debería encontrarse siempre entre 1 y 5.
- **Integridad referencial:** comprobar que los registros de las tablas relacionadas mantengan sus referencias válidas. Por ejemplo, que todo `id_cliente` presente en los casos exista también en la tabla `clientes`.
- **Reconciliación diaria:** comparar los principales totales y métricas del corte actual con los del día anterior para identificar variaciones anómalas que no tengan una explicación esperada.

Estos chequeos se implementan como validaciones SQL que corren como un paso explícito del pipeline, antes de publicar los datos nuevos. Si alguna validación falla, el pipeline se detiene, el tablero conserva el último dato válido conocido en lugar de mostrar un dato corrupto, y se dispara una alerta automática al equipo de datos. Es preferible que el tablero muestre datos de ayer con una alerta visible, a que muestre datos de hoy que están corruptos sin que nadie lo note.

Sección 5 - Inventario y migración de reportes existentes

5.1. Análisis y plan de migración

Recibe el siguiente inventario de 10 reportes existentes. Su tarea es analizarlos y definir un plan de migración a Power BI:

ID	Descripción	Último uso	Usuarios	Frecuencia	Fuente
R01	Casos por cliente y período	Hace 2 días	15	Diario	BD principal
R02	Casos por cliente y período v2	Hace 3 días	12	Diario	BD principal
R03	Costo acumulado del año	Hace 1 mes	3	Mensual	BD principal
R04	Costo acumulado año (copia)	Hace 8 meses	0	Mensual	BD principal
R05	Clientes sin actividad de prevención	Hace 1 día	20	Semanal	BD principal
R06	Indicadores ejecutivos	Hace 2 días	8	Diario	Vista legada
R07	Detalle de casos abiertos	Hace 1 semana	25	Diario	BD principal
R08	Análisis histórico 5 años	Hace 6 meses	2	Trimestral	Archivo Excel
R09	Facturación vs presupuesto	Hace 3 días	6	Mensual	BD principal + ERP
R10	Prueba indicadores nuevos	Hace 1 año	0	—	BD de pruebas

5.1.1. ¿Cuáles migra primero y por qué? ¿Cuáles descartaría sin migrar? Justifique cada decisión.

Dado que el criterio de priorización combina cuatro señales: si el reporte sigue en uso real (último uso reciente vs abandonado), cuántos usuarios dependen de él, qué tan crítica es su frecuencia operativa, y qué tan riesgosa es su fuente de datos, migraría primero:

- R07 (Detalle de casos abiertos): es el reporte con más usuarios de todo el inventario (25). Priorizarlo temprano significa que, si la migración revela algún problema, se detecta con tiempo de sobra antes de afectar a la mayor cantidad de gente posible.

- R05 (Clientes sin actividad de prevención): 20 usuarios, uso semanal muy activo (hace 1 día), y es un reporte directamente accionable para el negocio (prioriza visitas de prevención).
- R01 (Casos por cliente y período), una vez resuelta su duplicidad con R02.

Descartaría sin migrar:

- R04 (Costo acumulado año - copia): 0 usuarios y sin uso hace 8 meses, además de que parece ser una copia de R03. No hay evidencia de que alguien lo necesite.
- R10 (Prueba indicadores nuevos): 0 usuarios, sin frecuencia definida, y sale de una base de datos de pruebas. Es un artefacto de desarrollo, no un reporte de producción.

Migraría después, con trabajo adicional previo:

- R06 (Indicadores ejecutivos): activo y de alto perfil (gerencia), pero su fuente es una vista legada que requiere entender su lógica interna antes de poder fijar una fecha de migración confiable.
- R09 (Facturación vs presupuesto): activo, pero cruza BD principal con ERP, lo que añade complejidad de integración que exige más tiempo de diseño y pruebas.
- R03 (Costo acumulado del año): activo (uso mensual coherente con su frecuencia), pero con solo 3 usuarios, por lo que no compite por los primeros lugares de la fila.

Finalmente, pospondría:

- R08 (Análisis histórico 5 años): frecuencia trimestral, solo 2 usuarios, y viene de un archivo Excel, la fuente más riesgosa del inventario. No es urgente, y amerita una validación de calidad previa antes de migrarlo, no una migración apurada.

5.1.2. R01 y R02 parecen ser el mismo reporte. ¿Qué haría antes de migrar cualquiera de los dos?

Compararía la lógica de construcción de ambos reportes (consultas, filtros, agregaciones), para determinar si realmente calculan lo mismo o si difieren en algún criterio. También entrevistaría a los dos grupos de usuarios (15 y 12 personas respectivamente) para entender por qué conviven dos versiones: es posible que R02 se haya creado para corregir un problema de R01 sin que nadie deprecara el original, o que dos equipos hayan construido reportes similares de forma independiente.

Con esa información, definiría una única versión oficial (fusionando lo mejor de ambas si cada una resolvía algo distinto), comunicaría a los usuarios de la versión descartada hacia

dónde migrar, y solo entonces migraría la versión consolidada a Power BI. Migrar ambas por separado solo trasladaría el mismo problema de duplicidad y desconfianza a la herramienta nueva.

5.1.3. R06 consume una vista legada y R08 viene de Excel. ¿Qué riesgos anticipa en cada caso y cómo los manejaría?

- **R06 (vista legada):** el riesgo principal es que la vista probablemente encapsula lógica de negocio (joins, filtros, cálculos) que nadie ha documentado, y que no es evidente solo con mirar su nombre. Migrar sin entender esa lógica significa heredar silenciosamente cualquier problema que tenga, con el detalle particular de que es un reporte para gerencia, donde el costo de un error de datos es alto.
Antes de migrar, extraería y documentaría la definición completa de la vista (su DDL), documentando nombre, propósito, tablas fuente, campos calculados y notas de uso; y solo conectaría el modelo semántico a esa lógica una vez validada, nunca directamente a la vista sin revisión previa.
- **R08 (archivo Excel):** el riesgo principal es la ausencia de control de versiones y gobernanza: el archivo puede tener ediciones manuales, fórmulas rotas, o ser mantenido de forma informal por una sola persona (un punto único de fallo), sin garantía de que la versión entregada sea la correcta o la más reciente. Migrar un error histórico tal cual a Power BI le daría una falsa apariencia de validado, solo por estar en una herramienta más seria.
Antes de migrar, aplicaría un proceso de validación de calidad (completitud, validez, unicidad, consistencia, oportunidad) sobre los 5 años de histórico, entrevistaría a quien mantiene el archivo para entender su proceso de actualización, y evaluaría si ese histórico puede reconstruirse desde la base de datos principal en lugar de depender del Excel como fuente primaria.

Sección 6 - Criterio sobre IA aplicada y agentes conversacionales

6.1. Criterio sobre RAG y agentes

La organización quiere que los coordinadores de cuenta puedan preguntarle a un asistente en Teams cosas como: ¿cuántos casos tuvo el cliente X este mes? ¿Cuál es su nivel de riesgo? — sin necesitar abrir Power BI ni llamar al equipo de datos.

6.1.1. ¿Qué es una arquitectura RAG y cómo aplicaría ese concepto para resolver este caso? Explíquelo sin tecnicismos.

RAG (Retrieval-Augmented Generation) funciona como alguien que, antes de responder una pregunta, primero busca el dato exacto en el archivo correcto, en lugar de contestar de memoria y arriesgarse a equivocarse. Es un enfoque en el que el modelo de lenguaje no responde únicamente con lo que "sabe" internamente, sino que primero recupera información relevante de una fuente externa y confiable, y usa esa información para construir la respuesta.

Para este caso, si el coordinador pregunta en Teams "¿Cuántos casos tuvo el cliente X este mes?", el agente debe: interpretar la pregunta, identificar el cliente y el período solicitado, consultar la fuente de información autorizada de la organización, y usar ese resultado para responder en lenguaje natural. De esta manera, el modelo de lenguaje se encarga de entender la pregunta y redactar la respuesta, pero el dato siempre proviene de la fuente de negocio, no de lo que el modelo "recuerde". Esto reduce el riesgo de que el agente invente o distorsione información que no corresponde con los datos reales de la organización.

6.1.2. ¿Por qué conectar ese agente al modelo semántico de Power BI sería mejor que conectarlo directamente a las tablas de la base de datos? ¿Qué ventajas tiene para la consistencia de las respuestas?

Porque el modelo semántico centraliza las definiciones y reglas de negocio: qué se considera un caso, cómo se calcula la tasa de incidencia, o cómo se determina el nivel de riesgo de un cliente. Al conectar el agente a esa misma capa, en lugar de a las tablas crudas, sus respuestas quedan automáticamente alineadas con los mismos indicadores que ya usan los reportes y tableros de Power BI. Esto evita que el agente se convierta en una nueva fuente independiente que recalcula las métricas a su manera (como el caso de los 7 reportes con tasa de incidencia inconsistente).

6.1.3. ¿Cuál es el mayor riesgo de un agente conversacional sobre datos de negocio? ¿Cómo lo mitigaría?

El riesgo principal es que el agente entregue una respuesta incorrecta con un alto nivel de confianza. Esto puede ocurrir porque interprete mal la pregunta, no encuentre información suficiente, o genere una respuesta que no esté realmente respaldada por los datos.

Para mitigar este riesgo consideraría:

- Conectar el agente únicamente a fuentes gobernadas (el modelo semántico certificado), nunca a datos sin validar.
- Aplicar los mismos permisos de acceso que ya usan los usuarios (por ejemplo, el RLS por coordinador), para que el agente no exponga información que esa persona no debería ver.
- Definir explícitamente que el agente no debe inventar una respuesta cuando no encuentre datos suficientes, y que en ese caso debe decirlo abiertamente en lugar de generar una respuesta que suene bien, pero con información falsa.
- Probar el agente con un conjunto de preguntas y respuestas esperadas antes de liberarlo, y mantener monitoreo continuo de las interacciones reales.
- Mostrar trazabilidad de la fuente usada para cada respuesta, para que el usuario pueda verificarla si lo necesita.

Con estas medidas, el agente funciona como una interfaz de consulta sobre los datos ya gobernados de la organización, y no como una fuente independiente de información.

6.2. Experiencia o acercamiento con IA aplicada

Describa brevemente su experiencia o acercamiento con tecnologías de inteligencia artificial aplicada — modelos de lenguaje, agentes, automatización con IA u otras. No se busca un experto — se valora honestidad, disposición a aprender y criterio sobre cuándo aplicar estas herramientas y cuándo no.

6.2.1. ¿Qué herramientas o tecnologías de IA ha usado o explorado? ¿En qué contexto?

Mi experiencia más técnica es en la construcción de pipelines RAG (Retrieval-Augmented Generation). Para mi tesis de maestría, titulada "Detección automatizada de habilidades mediante modelos grandes de lenguaje (LLM) y recuperación aumentada (RAG)", utilicé un modelo open source de la familia Gemma de Google (gemma-3-4b-it) para extraer habilidades a partir de descripciones de ofertas de trabajo, y un modelo de embeddings

(embeddinggemma-300m) para construir un módulo de normalización: un sistema basado en RAG que mapea esas habilidades, extraídas en lenguaje natural, hacia la taxonomía estructurada de clasificación de habilidades laborales de la Comisión Europea (ESCO). Invoqué ambos modelos mediante la librería Transformers de Hugging Face, en un entorno con aceleración por GPU.

También he utilizado herramientas de IA generativa de propósito general, como ChatGPT, Claude, Copilot y Gemini, tanto en contextos profesionales como personales. Principalmente las he utilizado como asistentes para explorar ideas, resolver problemas, generar y revisar código, analizar información y acelerar tareas de investigación y aprendizaje. Esto me ha permitido incorporarlas como herramientas de apoyo a mi trabajo, teniendo en cuenta cuándo aportan valor frente a soluciones más convencionales

6.2.2. Si tuviera que construir un primer prototipo de agente conversacional sobre los datos de esta prueba, ¿por dónde empezaría? ¿Qué herramienta elegiría y por qué?

Empezaría por algo simple que pueda demostrar valor real, en lugar de una arquitectura más sofisticada: Conectaría un agente directamente a un modelo semántico de Power BI certificado, dado que el modelo semántico ya centraliza las definiciones de negocio (una sola definición de "tasa de incidencia", medidas certificadas, RLS por coordinador), así que el agente hereda esa confiabilidad en lugar de tener que reconstruirla.

En términos de herramienta, elegiría un agente basado en un modelo de lenguaje con capacidad de *function calling* o *tool-calling*, conectado al modelo semántico de Power BI. Esto permitiría que el agente interprete la pregunta del usuario y utilice como herramienta una consulta al endpoint XMLA, o una API intermedia que traduzca la pregunta en lenguaje natural a una consulta DAX o en la ejecución de una medida ya definida.

Para un primer prototipo, considero este enfoque más adecuado que un pipeline RAG tradicional (embeddings + base vectorial), porque aquí la información ya está estructurada y gobernada dentro del modelo semántico. El reto principal no es encontrar información relevante en documentos, sino traducir correctamente la pregunta del usuario a una consulta sobre los datos existentes, manteniendo las definiciones de negocio y las reglas de acceso ya establecidas.

6.2.3. ¿En qué situaciones NO usaría un agente conversacional y preferiría otro tipo de solución?

No usaría un agente conversacional cuando la pregunta requiera precisión numérica garantizada y auditable, como cálculos financieros o reportes regulatorios, donde un modelo de lenguaje puede introducir error incluso con buenas herramientas de soporte, y donde un reporte tradicional con la lógica validada es más confiable que una respuesta generada. Tampoco lo usaría para decisiones de alto impacto sin que haya una revisión humana (por ejemplo, decisiones que afecten la cobertura o el riesgo asignado a un cliente o trabajador individual), dado el riesgo de que el modelo generalice mal un caso particular. Y no lo usaría como reemplazo de un tablero bien diseñado para necesidades de monitoreo recurrente: si alguien necesita ver los mismos 5 indicadores todos los días, un tablero fijo es más rápido y predecible que formular la misma pregunta en lenguaje natural cada día.